

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 2 Exam Notes: Moving from C to C++

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	C++ Streams, Namespaces & Operators	2
1.1	Stream-Based Input/Output	2
1.2	Namespaces	2
1.3	Scope Resolution Operator (::)	2
1.4	Variable Declaration and References	3
2	Parameter Passing & Functions	3
2.1	Parameter Passing Mechanisms	3
2.2	Inline Functions	4
2.3	Function Overloading	5
2.4	Default Arguments	5

1. C++ Streams, Namespaces & Operators

C++ maintains backwards compatibility with C but introduces modern syntax and strict type checking to simplify development.

1.1 Stream-Based Input/Output

Unlike C which uses formatted functions like `printf` and `scanf`, C++ implements stream-based I/O. Streams are continuous sequences of bytes representing data flow.

- **cout:** Standard output stream (connected to console), used with the stream insertion operator (`<<`).
- **cin:** Standard input stream (connected to keyboard), used with the stream extraction operator (`>>`).
- **Type Safety:** Operator overloading allows streams to automatically detect variables' data types (no format specifiers like `%d` or `%f` required).

1.2 Namespaces

Namespace: A declarative region that provides a scope to the identifiers (names of types, functions, variables, etc.) inside it. Namespaces are used to organize code into logical groups and prevent name conflicts (clashing) in large projects.

The C++ Standard Library is defined within a namespace named `std`. To use identifiers like `cout` or `cin`, we must specify their namespace:

using scope resolution: `std::cout << "Hello";` (1)

using declaration: `using std::cout; cout << "Hello";` (2)

using directive: `using namespace std; cout << "Hello";` (3)

1.3 Scope Resolution Operator (::)

In C, if a local variable hides a global variable with the same name, there is no direct way to access the global variable. C++ introduces the scope resolution operator (`::`) to resolve scope conflicts.

Scope Resolution Syntax

Global Variable Access: `::variable_name` (4)

It is also used for:

- a. Accessing static members of a class: `ClassName::static_member`.
- b. Defining class member functions outside the class declaration.
- c. Resolving namespace paths: `NamespaceName::identifier`.

Code Example: Global variable access via Scope Resolution

```
#include <iostream>
int x = 10; // Global variable
int main() {
    int x = 20; // Local variable hiding global x
    std::cout << "Local x = " << x << std::endl;    // Prints 20
    std::cout << "Global x = " << ::x << std::endl;    // Prints 10
    return 0;
}
```

1.4 Variable Declaration and References

- **Declaration at Point of Use:** C requires variable declarations at the start of a block. C++ allows declaring variables anywhere in the code, immediately before their first use.
- **Reference Variables (Aliases):**

Reference Variable: An alias or alternative name for an already existing variable. It shares the same memory space as the original variable. Modifying the reference modifies the original variable.

Declaration Syntax:

```
int x = 10; int &y = x; // y is a reference to x
```

 (5)

Reference variables must be initialized during declaration and cannot be re-bound to another variable.

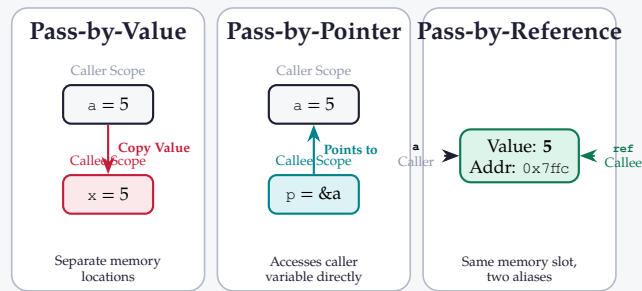
2. Parameter Passing & Functions

C++ extends function properties by introducing call-by-reference, inline compilation, and parameter defaults.

2.1 Parameter Passing Mechanisms

1. **Pass-by-Value:** A copy of the actual argument is passed to the function parameter. Changes inside the function do not affect the original variable.
2. **Pass-by-Pointer:** The address of the argument is passed. The function dereferences the pointer to access/modify the original variable.
3. **Pass-by-Reference:** The function parameter becomes an alias (reference) to the actual argument. No new memory is allocated, and the function directly manipulates the caller's variables.

Memory Behavior of Parameter Passing



Code Example: Swapping variables using references

```
#include <iostream>
void swap(int &x, int &y) { // x and y are references to caller variables
    int temp = x;
    x = y;
    y = temp;
}
int main() {
    int a = 5, b = 10;
    swap(a, b); // Swaps values of a and b directly
    std::cout << "a = " << a << ", b = " << b; // Prints a = 10, b = 5
    return 0;
}
```

2.2 Inline Functions

Function calls introduce overhead: pushing arguments onto stack, saving registers, jumping to function address, restoring state on return. For small functions, the overhead can exceed the execution time of the function body.

Inline Function: A function that is expanded in line at the point of call by the compiler instead of being invoked via standard function call stack jumps.

Syntax:

```
inline int add(int a, int b) { return a + b; } (6)
```

Key Rules and Compiler Discretion: The `inline` keyword is a **request**, not a command. The compiler can ignore it and compile the function normally if:

- The function contains loops (`for`, `while`), `switch` statements, or `goto`.
- The function is recursive.
- The function contains static variables.
- The function returns a value but does not contain a `return` statement.

2.3 Function Overloading

Function overloading is a compile-time polymorphism feature allowing multiple functions to share the same name with different parameters.

Function Overloading: The capability to declare multiple functions with the same name within the same scope, provided their signatures (parameter list) are unique.

Resolution Criteria: The compiler distinguishes overloaded functions based on:

1. **Number of parameters:** e.g., `void display(int)` vs `void display(int, int)`.
2. **Type of parameters:** e.g., `void display(int)` vs `void display(double)`.
3. **Order of parameters:** e.g., `void display(int, double)` vs `void display(double, int)`.

Note: Functions cannot be overloaded based solely on their return types.

2.4 Default Arguments

Default arguments allow functions to assign a default value to a parameter if the caller does not provide an argument for it. **Syntax:**

```
void printLine(char c = '*', int len = 40); // default values (7)
```

Critical Rule: Default arguments must be assigned from **right to left**. We cannot assign a default value to a parameter unless all parameters to its right also have default values.

- **Correct:** `void func(int a, int b = 10, int c = 20);`
- **Incorrect:** `void func(int a = 10, int b, int c = 20);` // b lacks default

Quick Revision — Module II

Moving from C to C++ | OE-EC604C

Key Formulas and Syntax

Reference Variable	<code>int x = 5; int &ref = x; // ref shares memory with x</code>
Inline Function	<code>inline int square(int x) { return x * x; }</code>
Scope Resolution	<code>::variable</code> (accesses global variable hiding in local scope)
Default Parameters	<code>void value(int p, int n = 5, float r = 0.1); // right-to-left</code>

One-Line Definitions

- **Reference Variable:** A constant pointer to variable that acts as an alternative name (alias) for that variable.
- **Namespace:** A scope region containing definitions to prevent name clashes in large programs.
- **Scope Resolution (::):** An operator used to access global variables or define class methods externally.
- **Inline Function:** A function that is compiled by replacing its calls directly with the function code.
- **Function Overloading:** Declaring multiple functions sharing the same name but unique parameters.
- **Default Argument:** A value assigned to a function parameter in its prototype used if argument is omitted.
- **Pass-by-Reference:** Passing parameters by sharing the caller's variable memory directly via references.
- **Type Safety:** Strict compile-time checks verifying variable types and function arguments matching.

Frequently Asked / Viva Questions

Q1. Why does C++ input/output streams compile with greater type safety than C standard I/O?

C++ streams use overloaded insertion/extraction operators to automatically detect variable types, removing format specifier errors.

Q2. Can we declare reference variables without initialization in C++?

No. A reference must be initialized to bind to a variable during declaration. E.g., `int &ref;` causes a compile error.

Frequently Asked / Viva Questions (contd.)**Q3. Can a reference variable be re-assigned to reference another variable later?**

No. Once a reference variable is initialized to a target variable, it cannot be changed to reference any other variable.

Q4. How does the scope resolution operator access global variables?

Adding the `::` prefix tells the compiler to search for the identifier in the global scope instead of the local stack block.

Q5. What is the primary benefit of using inline functions?

It eliminates function call overhead (saving registers, jumping, stack frames), improving execution speed for small functions.

Q6. Under what conditions will the C++ compiler ignore an inline request?

If the function is large, recursive, contains loops (`for`, `while`), `switch/goto` statements, or static variables.

Q7. Can we overload a function based only on its return type?

No. The compiler resolves overloading by comparing parameter lists (number, type, order). Return types are ignored.

Q8. State the mandatory rule for declaring default arguments in C++.

Default arguments must be added from right to left in the parameter list. No default argument can precede a non-default one.

Q9. What is the difference between pass-by-pointer and pass-by-reference?

Pass-by-pointer passes a memory address (can be NULL, requires dereferencing). Pass-by-reference passes an alias (cannot be NULL, behaves like local variable).

Q10. Explain the use of the `using namespace std;` statement.

It imports all identifiers from the standard library namespace (`std`) into global scope, removing the need to write `std::` prefixes.

Q11. Why are inline functions preferred over preprocessor macros (`#define`)?

Inline functions are compiled with full type-checking and follow scope and access rules, whereas macros are simple text replacements.

Q12. What is a name collision/clash and how do namespaces solve it?

A name collision occurs when two libraries define classes or functions with identical names. Namespaces solve this by wrapping definitions in unique scope scopes.

Comparison of Parameter Passing Methods

Parameter	Pass-by-Value	Pass-by-Pointer	Pass-by-Reference
Memory Allocation	Allocates new local storage for parameter copy.	Allocates storage for pointer address (4/8 bytes).	No new memory allocated. Shares caller's slot.
Null Values	Not applicable.	Pointer can be <code>nullptr</code> .	Reference cannot be null.
Syntax	Simple: <code>func(a)</code> .	Complex: <code>func(&a)</code> and <code>*ptr</code> .	Simple: <code>func(a)</code> with parameter <code>&ref</code> .
Modifies Caller?	No, operations are on copy.	Yes, via pointer dereferencing.	Yes, directly via alias.